School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

Fall Semester 2026-27

ASSESSMENT IX – PROJECT REVIEW 2

DagOpt: DAG-Based Optimization of Basic Blocks

Common Subexpression and Redundant Computation Elimination using Directed Acyclic Graphs

Programme & Branch	B.Tech (CSE & Specialization) & SCOPE
Course Name & Code	Compiler Design Lab (BCSE307P)
Class Number	VL2026270103361
Slot	L15+L16
Faculty Name	Dr. Bhawana Tyagi
Review Date	14/10/2026

Team Members

S.No	Name	Register Number
1	Harshit Pandey	24BCT0173
2	Ankush Kumar	24BCT0254
3	Lakshya Kulshrestha	24BCT0299

1. Introduction and Progress Overview

Code produced by a compiler front end repeats computations: the same expression is evaluated several times and values are recomputed although their operands have not changed. DagOpt removes this redundancy inside basic blocks. It builds a directed acyclic graph (DAG) for every basic block, in which each distinct value is one node and every variable holding that value is a label on the node. A computation that already has a node is not repeated, which eliminates common subexpressions.

The objectives of the project are: (1) parse three-address code (TAC); (2) split it into basic blocks; (3) build a DAG per block with sharing of identical computations, including commutative operands; (4) eliminate common subexpressions and redundant computations; (5) regenerate optimized TAC from the DAG and compare it with the original; (6) verify that the optimization preserves meaning.

The front end (parser, block splitter and DAG construction with common subexpression detection) was completed earlier. This report describes the work done since then: simplification of constants and algebraic identities inside the DAG, liveness analysis, **regeneration of optimized TAC from the DAG**, semantic verification, DAG visualization, metrics and a command-line tool. The status of every stage is shown below.

Stage	Name	Purpose	Status
1	TAC parsing	Read the .tac file into a list of instructions	Completed
2	Basic block splitting	Divide instructions into basic blocks at leaders and build the control flow edges	Completed
3	DAG construction	One DAG per block with sharing (CSE), constant folding and algebraic identities	Completed
4	Optimized code output	Regenerate TAC from the DAG with liveness-based dead code removal	Completed
5	Verification and analysis	Random-input equivalence checking, metrics, DOT export	Verification, metrics and DOT export completed; large-scale analysis remaining

2. System Architecture and Implementation

The tool is a Python 3.10+ package (dagopt) with one module per responsibility, so every stage is tested on its own. The input of each stage is the output of the previous one.

2.1 Module Summary

Module	Responsibility
tac.py	Instruction dataclass, parser, formatter, syntax errors; safe_div and safe_mod with C-style truncation toward zero; temporary-name test
blocks.py	Leader detection, block splitting, control flow graph edges (successors and predecessors)

Module	Responsibility
dag.py	DAG nodes and construction; hash-based value sharing; commutative canonicalization; counters for CSE, folding and algebraic simplifications; live node selection
simplify.py	Constant folding and algebraic identities; each result carries a tag (fold or alg)
liveness.py	Live-out variables of a block, including variables read by the terminating branch
codegen.py	Regeneration of TAC from the DAG: naming, copy generation, dependency scheduling, cycle breaking, terminator rewriting
interp.py	Interpreter for TAC blocks over concrete values
verify.py	Random-input equivalence check of original and optimized blocks; random block generator for fuzzing
dotexport.py, tools/render_dag.py	Graphviz DOT export and PNG rendering of each block DAG
metrics.py	Per-block and total statistics, printed as a table and exported as JSON and CSV
main.py, cli.py	Command-line driver that runs the whole pipeline

2.2 Input Format

Each non-blank, non-comment line is one of: x = y (copy or integer), x = - y (negation), x = y OP z with OP in + - * / %, a label L:, goto L, and if a REL b goto L with REL in < <= > >= == !=. Text after # is a comment.

2.3 DAG Construction with Simplification

Instructions of a block are processed in order. A table maps each variable to the node that currently holds its value, and a hash table maps a key (operator, left node, right node) to an existing node. If the key exists, the node is reused and the hit is counted as a common subexpression. For + and * the two operand nodes are put in a fixed order, so a + b and b + a give the same key. Because keys are built from node identities and not from variable names, a variable that is reassigned automatically refers to a new node, which keeps the sharing safe.

```
for each instruction x = y OP z:
    l = node_for(y); r = node_for(z)
    result = try_simplify(OP, l, r)
    if result is a constant fold (tag "fold") and folding is enabled:
        bind x to the constant node; count it
    elif result is an algebraic identity (tag "alg") and algebra is enabled:
        bind x to the surviving node or constant; count it
    else:
        key = (OP, l, r)   # (min, max) order for + and *
        if key in table: reuse node (CSE hit) else create node
        bind x to the node
```

The simplifier folds constants for neg, +, -, *, / and % (division and modulo truncate toward zero as in C, and a division by zero is never folded). It applies the identities x + 0 = x, 0 + x = x, x * 1 = x, 1 * x = x, x * 0 = 0, 0 * x = 0, x - 0 = x, x - x = 0 and x / 1 = x. The identity x / x

= 1 is intentionally not used, because it is wrong when x is 0. Every result is tagged as fold or alg, so the options --no-fold, --no-algebra and --no-commute switch each optimization off independently and each is counted separately.

2.4 Liveness and Dead Code

A node is needed only if its value can be observed after the block. The live-out set of a block contains every program variable that appears in the block (temporaries are excluded, see the temporary-name pattern in Section 2.7) and every variable read by the terminating branch. A different set can be supplied with --live-out. All nodes reachable from the nodes of live variables are live; the other nodes are dead and are not emitted. This is what removes the unused temporaries.

2.5 Code Generation from the DAG

Code generation walks the live nodes and produces instructions in four steps.

1. **Naming.** Leaves keep their variable name or constant. An operation node is named by one of its live labels, preferring a program variable over a temporary; if it has no label, a fresh temporary _n1, _n2, ... is created.
2. **Emission.** One instruction is emitted per live operation node. Every other live label of the node becomes a copy from the node name, for example y = x. This is how a repeated computation turns into a cheap copy.
3. **Scheduling.** An instruction that uses a value computed in the block must come after the instruction that computes it. An instruction that reads the original value of a variable must come before the instruction that overwrites that variable. Ready instructions are emitted in node creation order. An instruction that reads and writes the same variable, such as i = i + 1, needs no special handling.
4. **Cycle breaking.** If the dependencies form a real cycle, as in a swap, the original value of one variable is first copied to a fresh temporary and the reader uses the temporary. Leading labels of the block are written first, and the operands of the final conditional branch are rewritten to the names chosen for their nodes.

2.6 Verification

Equivalence is checked by testing. The interpreter executes the original block and the optimized block from the same random initial values, with C-style division and modulo, and compares all live-out variables. The check is run for a chosen number of trials per block (--verify N, with --seed for repeatability). A random block generator is used to fuzz the optimizer.

2.7 Command-Line Tool

The tool is run as: python -m dagopt.main input.tac [options].

Option	Meaning
-o FILE	Write the optimized TAC to a file

Option	Meaning
--live-out a,b	Variables treated as live after every block
--temp-regex R	Pattern for temporaries (default: t followed by digits, or _n followed by digits)
--no-fold, --no-algebra, --no-commute	Switch off constant folding, algebraic identities or commutative sharing
--dot-dir DIR	Write one Graphviz DOT file per block
--verify N, --seed S	Random equivalence testing with N trials per block
--metrics, --metrics-json, --metrics-csv	Print the statistics table and export it as JSON and CSV

3. Results

All results below were produced by the tool with --metrics --verify 500 --seed 1. InstrIn and InstrOut count the instructions of a block without its labels.

3.1 Worked Example

Input TAC	Optimized TAC

`t1 = a + b`
`t2 = a + b`
`t3 = b + a`
`x = t1 * c`
`y = t2 * c`
`z = t3 + x`
	`t1 = a + b`
	`x = t1 * c`
	`y = x`
	`z = t1 + x`

Node	Operator	Children	Labels	Note
n0	var	–	a	Leaf
n1	var	–	b	Leaf
n2	+	n0, n1	t1, t2, t3	t2 and t3 reuse n2 (t3 by commutativity)
n3	var	–	c	Leaf
n4	*	n2, n3	x, y	y reuses n4
n5	+	n2, n4	z	New computation

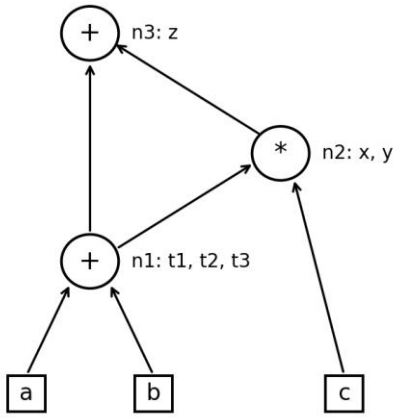


Figure 1: DAG of the worked example (circles are operations, boxes are leaves)

The six arithmetic operations of the input need only three in the output: three computations were common subexpressions. The copies t2 = t1 and t3 = t1 are removed as well, because t2 and t3 are temporaries that are never used after the block. The block shrinks from 6 to 4 instructions and the equivalence check passes.

3.2 Common Subexpression Sample

Input TAC	Optimized TAC

`a = b + c`
`d = b + c`
`e = a * d`
	`a = b + c`
	`d = a`
	`e = a * a`

3.3 Constants and Algebraic Identities

Input TAC	Optimized TAC

`a = 3`
`b = 4`
`c = a + b`
`d = x * 0`
`e = y - y`
`f = z + 0`
	`a = 3`
	`b = 4`
	`c = 7`
	`d = 0`
	`e = 0`
	`f = z`

All four arithmetic operations disappear: a + b is folded to 7 (one constant folding), and x * 0, y - y and z + 0 are removed by algebraic identities (three simplifications).

3.4 Loop with a Branch

Input TAC	Optimized TAC

`i = 0`
`L1:`
`a = x + y`
`b = x + y`
`i = i + 1`
`if i < 10 goto L1`
`res = a + b`
	`i = 0`
	`L1:`
	`a = x + y`
	`b = a`
	`i = i + 1`
	`if i < 10 goto L1`
	`res = a + b`

The program has three blocks. The repeated x + y inside the loop block is removed and the label L1 and the branch are preserved. The increment i = i + 1 stays a single instruction.

3.5 Swap (Cycle Breaking)

Input TAC	Optimized TAC

`t = a`
`a = b`
`b = t`
	`t = a`
	`_n1 = a`
	`a = b`
	`b = _n1`

Here a and b exchange values, so the instructions depend on each other in a cycle. The generator saves the original value of a in the fresh temporary _n1 and the result is verified correct. It uses one more instruction than the input; a smaller output is possible and is listed as future work.

3.6 Summary of Measurements

Program	Instr. before	Instr. after	Arith. before	Arith. after	CSE hits	Folded	Algeb.	Verif.
Worked example	6	4	6	3	3	0	0	True
CSE sample	3	3	3	2	1	0	0	True
Constants sample	6	6	4	0	0	1	3	True
Loop sample	6	6	4	3	1	0	0	True
Swap sample	3	4	0	0	0	0	0	True
T01	4	4	4	3	1	0	0	True
T02	3	2	3	2	1	0	0	True
T05	6	6	4	0	0	4	0	True
T06	6	6	4	0	0	4	0	True
T07	3	3	2	2	0	0	0	True

The number of arithmetic operations is the main measure of improvement. A redundant computation is replaced by a copy, so the instruction count does not always drop, but the arithmetic count does in every program that has redundancy. Across the ten programs above, 34 arithmetic operations in the inputs become 15 in the outputs, and every output passes the random equivalence check.

3.7 Testing

The test suite has 36 tests and all of them pass: test_tac 12, test_codegen 5, test_interp 4, test_blocks 3, test_dag 3, test_simplify 3, test_verify 3, test_dotexport 1, test_e2e 1 and test_metrics 1. The tests cover the parser, block splitting, DAG sharing, simplification (including that x / x is not simplified), dead code removal, cycle breaking, label preservation and the interpreter. The end-to-end test runs the whole pipeline on the test programs T01, T02, T05, T06 and T07.

4. Novelty

- **Tagged simplifications with independent switches.** Constant folding, algebraic identities and commutative sharing can each be disabled, and each has its own counter. This allows the contribution of every technique to be measured separately on the same program.
- **Dependency-aware code generation.** The generator reads each variable before it is overwritten, uses a copy to a fresh temporary only when a real cycle exists, and keeps labels and the operands of the final branch correct. A DAG scheduled in an arbitrary order would give wrong code in these cases.
- **Branch-aware liveness.** Variables read by the terminating branch are always kept alive, so the loop condition is never optimized away.
- **Built-in semantic verification.** The tool does not only report counts; it checks each optimized block against the original with an interpreter that follows C division and modulo, using random inputs and a random block generator.
- **Inspectable output.** The DOT export shades nodes shared by several users (the common subexpressions) and draws dead nodes dashed, and the metrics are exported as a table, JSON and CSV.

5. Limitations and Remaining Work

Current limitations:

- Liveness is local to a block. All program variables of a block are assumed live at its end, so an assignment that is dead because of later blocks is kept. A data-flow analysis over the control flow graph would remove it.
- The equivalence check is by random testing of each block; it is not a formal proof.
- Strength reduction (for example x * 2 as x + x) and double negation are not implemented.
- The swap sample uses one more instruction than necessary.

Work remaining for the final review: global liveness analysis over the control flow graph, further identities, a smaller output for cycles, a comparison of the options (all optimizations, without folding, without algebra, without commutativity) on a larger set of programs, and the final analysis and report.

References

1. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson/Addison-Wesley, 2006.
2. J. Cocke and J. T. Schwartz, Programming Languages and Their Compilers: Preliminary Notes. Courant Institute, New York University, 1970.

3. A. V. Aho and J. D. Ullman, "Optimization of straight line code," SIAM Journal on Computing, vol. 1, no. 1, pp. 1–19, 1972.
4. A. V. Aho, S. C. Johnson and J. D. Ullman, "Code generation for expressions with common subexpressions," Journal of the ACM, vol. 24, no. 1, pp. 146–160, 1977.
5. P. Briggs, K. D. Cooper and L. T. Simpson, "Value numbering," Software: Practice and Experience, vol. 27, no. 6, pp. 701–724, 1997.
6. K. D. Cooper and L. Torczon, Engineering a Compiler, 2nd ed. Morgan Kaufmann, 2011.
7. S. S. Muchnick, Advanced Compiler Design and Implementation. Morgan Kaufmann, 1997.
8. C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in Proc. IEEE/ACM CGO, 2004, pp. 75–86.